

Senior Backend Engineer - Technical Test

- **Time-box:** ~3 hours
- **Stack:** Go + Node.js + Next.js 16.2.4 (provided)
- **External data source:** [The Odds API v4](#)
- **Swagger:** [API reference](#)
- **API key (provided):** 1408e341969941181bc487e2a9b671e2
- **Repository to clone & work in:** <https://github.com/flotaro/be-test> (monolith: UI + both services live here)

1. The Goal

You are joining a team building a sports-book front-end. The **Next.js 16.2.4 UI is already in the repo** under `/ui` and is wired against a defined backend contract. Your job is to build the **backend** that feeds the UI by **consuming The Odds API** and adding a small bet/wallet layer of your own.

You **must use both Go and Node.js**. We want to see *how you decide* which language fits which workload. Both reasonable splits will pass - what matters is the justification.

2. Repository Layout (already scaffolded)

Path	What it is
be-test/	root of flotaro/be-test
— README.md	this document
— ui/	Next.js 16.2.4 - pre-built. Do NOT modify the page code (you only set its env vars).
— services/	
— go-svc/	Go skeleton (go.mod, main.go, Dockerfile) - your code goes here
— node-svc/	Node skeleton (package.json, src/, Dockerfile) - your code goes here
— data/	
— users.json	250 seeded users for login. PIN = 1234 for all of them.
— notifications.json	400 seeded notifications (load into memory at startup)
— docker-compose.yml	stub bringing up: ui + go-svc + node-svc
— .env.example	contains ODDS_API_KEY=1408e341969941181bc487e2a9b671e2

You do **not** need a database. Keep users / wallet / bets / notifications **in-memory**, seeded from `data/*.json` at startup. If you really want Postgres or Redis you may add it to `docker-compose.yml`, but it is not required.

2.1 IMPORTANT: the UI ships with a mock layer - you must turn it off

So that we (the reviewers) and you can click through the UI before any backend exists, the Next.js app under `/ui` ships with a tiny **client-side mock** that returns canned data for every endpoint in §7.

Today, out of the box, `ui/.env.local` contains:

```
NEXT_PUBLIC MOCK=true
```

While this flag is `true`, the UI is **NOT talking to any backend** - it is serving canned sports, events, odds, wallet, bets and notifications from `ui/src/lib/mock.ts`. Login accepts any email + PIN. Placing a bet only updates the browser's in-memory state and resets on refresh.

Your job is to replace that mock with real services. Before you submit:

1. Edit `ui/.env.local` and set `NEXT_PUBLIC MOCK=false` (or delete the line).
2. Point `NEXT_PUBLIC WALLET_API` and `NEXT_PUBLIC PROFILE_API` at your two running services.
3. Verify every screen in the UI works against your backend (login, sports list, event odds, place bet, see it settle, balance updates, notifications).

We will grade you with `NEXT_PUBLIC MOCK=false`. **If the UI still works only because the mock is on, you fail the integration portion of the rubric (§10).** Do not modify `mock.ts` to make the UI "pass" - just turn it off.

If you want to preview the UI behaviour while you build, leave the flag on temporarily. Just remember to flip it before you push.

3. What the Frontend Needs

The provided Next.js UI has six screens. They are already coded against the contract below - your services must serve it.

Screen	What it does	Talks to
Login	Email + PIN → returns a JWT	Profile service
Sports	List sports from The Odds API	Odds service
Sport detail	List upcoming events for a sport	Odds service
Event detail	Show bookmaker odds for one event + bet slip	Odds service + Wallet
My Bets	The user's bet history	Wallet service
Wallet + Notifications	Balance, ledger, notification list	Wallet + Profile

Full API contract in §6.

4. What to Build

Two services. **You decide** which language for which.

Service A - Odds Proxy + Bet/Wallet Engine

Consumes The Odds API and is the source of truth for bets & wallet.

- `GET /sports` → proxies `GET /v4/sports` (cache 5 min - odds API has tight quotas)
- `GET /sports/:key/events` → proxies `GET /v4/sports/{sport}/events` (cache 30 s)

- `GET /events/:eventId/odds?sport=...` → proxies `GET /v4/sports/{sport}/events/{eventId}/odds?markets=h2h®ions=eu` (cache 15 s)
- `POST /bets` → debits wallet, stores bet
- `GET /bets?userId=...&limit=...` → bet history
- `GET /wallet/:userId` → balance + last 10 ledger entries
- `POST /bets/:id/settle` (internal worker fires this - see §5)

Hard requirements - Cache Odds API responses (you choose: in-memory LRU, TTL map, Redis - justify).
 - Wallet operations must be **concurrent-safe** - no negative balance, no double-spend. Prove it with a test that fires N parallel bets at one user.
 - Track quota: include the remaining quota header returned by The Odds API in your logs.

Service B - Profile + Notifications

- `POST /auth/login` - email + pin → JWT (24 h)
- `GET /me` - returns the logged-in player from the JWT
- `GET /notifications?limit=...` - paginated, newest first
- `POST /notifications/:id/read` - mark as read

Hard requirements - JWT validation on every protected endpoint. - Both services share the same JWT secret (env var).

5. The Decision You Must Make

Pick a language per service and write ≤ 200 words in your README explaining why. Look at the workloads:

- One service is **throughput-sensitive and transactional** - concurrent bets, in-memory ledger integrity, low-latency wallet reads.
- One service is mostly **I/O and JSON shuffling** - auth, listing notifications, returning the current user.

The Odds proxy *could* live in either; some candidates put proxying on the Node side (fast iteration, easy JSON) and concurrency-critical wallet logic on the Go side. Others put everything throughput-related (proxy + wallet) on Go. **Either is acceptable** - defend the choice.

The two services talk to each other over **HTTP** by default. If you prefer events / a queue, justify it.

6. Settlement Worker

When a bet is placed, simulate settlement: **30 seconds later** resolve with a 45% win chance. On a win, credit `stake * odds`.

Implement however you like - goroutine, Node interval, internal cron, lightweight queue. State the choice in the README.

7. API Contract (the UI depends on these - do not rename)

Endpoint	Auth / Request body	Response shape
<code>POST /auth/login</code>	<code>{ email, pin }</code>	<code>{ token, userId }</code>

Endpoint	Auth / Request body	Response shape
GET /me	Bearer	{ id, email, displayName, currency }
GET /sports	Bearer	[{ key, group, title, active, hasOutrights }]
GET /sports/:key/events	Bearer	[{ id, sportKey, commenceTime, homeTeam, awayTeam }]
GET /events/:eventId/odds?sport=...	Bearer	{ id, sportKey, commenceTime, homeTeam, awayTeam, bookmakers: [{ key, title, markets: [{ key, outcomes: [{ name, price }] }] }] }
POST /bets	Bearer { userId, eventId, sportKey, market, selection, stake, odds }	{ betId, status: "PENDING" }
GET /bets?userId=...&limit=...	Bearer	[{ id, eventId, market, selection, stake, odds, status, payout, placedAt, settledAt }]
GET /wallet/:userId	Bearer	{ balance, currency, ledger: [{ id, type, amount, createdAt }] }
GET /notifications?limit=...	Bearer	[{ id, title, body, readAt, createdAt }]
POST /notifications/:id/read	Bearer	{ ok: true }

All responses JSON. CORS open for `http://localhost:3000`.

The UI does not know which service is Go vs Node - it points at two URLs you choose in `.env`:

```
NEXT_PUBLIC_WALLET_API=http://localhost:8081    # whichever service owns Service A
NEXT_PUBLIC_PROFILE_API=http://localhost:8082   # whichever service owns Service B
```

8. The Odds API - Quick Notes

Base URL: `https://api.the-odds-api.com/v4` Auth: append `?apiKey=...` You will use these endpoints:

```
GET /v4/sports                list of sports
GET /v4/sports/{sport}/events  upcoming events for a sport
GET /v4/sports/{sport}/events/{eventId}/odds?regions=eu&markets=h2h  odds for an event
```

Sample sport keys to test with: `soccer_epl`, `basketball_nba`, `tennis_atp_french_open`, `americanfootball_nfl`. **Cache aggressively** - the free tier has tight monthly quotas. The API returns `x-requests-remaining` / `x-requests-used` headers - log them.

9. Deliverables

Push to a branch on `flotaro/be-test` (or fork it). Include:

1. Working code in `services/go-svc` and `services/node-svc`.
 2. `docker compose up` brings up: ui (port 3000), go-svc, node-svc. The UI must be usable end-to-end.
 3. README updates:
 4. Your **Go vs Node decision and reasoning** (≤ 200 words).
 5. Your **inter-service comms choice** and why.
 6. Your **caching strategy** for Odds API.
 7. What you would do next given another day.
 8. Tests:
 9. Wallet: **concurrency test** proving no overspend under N parallel bets.
 10. Profile: JWT validation + happy-path notifications test.
-

10. Evaluation Rubric

Area	Weight
Correct Go vs Node split, clearly reasoned	20%
Concurrent-safety of the wallet (provable by your test)	25%
Odds API integration: works, cached, quota-aware	20%
API contract matches §7 and the UI works end-to-end	15%
Code quality, idiomatic per language	10%
<code>docker compose up</code> works first try	10%

11. What We Are NOT Looking For

- A real database (in-memory is fine).
 - Full sportsbook semantics - h2h market only is enough.
 - Front-end changes.
 - 100% coverage - we want **the right tests**, not many.
 - Microservices ceremony.
-

12. Suggested Time Budget

Read brief, sketch the split	15 min
Docker + skeletons running, env wired	25 min
Service A - Odds proxy + caching	40 min
Service A - bets + wallet + concurrency test	50 min
Service B - auth + profile + notifications	30 min

End-to-end smoke test in UI, write README decisions	20 min
---	--------

Cut scope if you fall behind - say so in the README. One polished service beats two half-done ones.
Good luck.